

Screenshots:

```
3. Exploratory Data Analysis (EDA)

# Dataset shape: rows and columns.
df.shape

(150, 5)

# Data types for every column.
df.dtypes

...
0
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
species              category
dtype: object
```

```
# Null value check.
df.isnull().sum()

0
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
species              0
dtype: int64

# Descriptive statistics for numerical features.
df.describe()

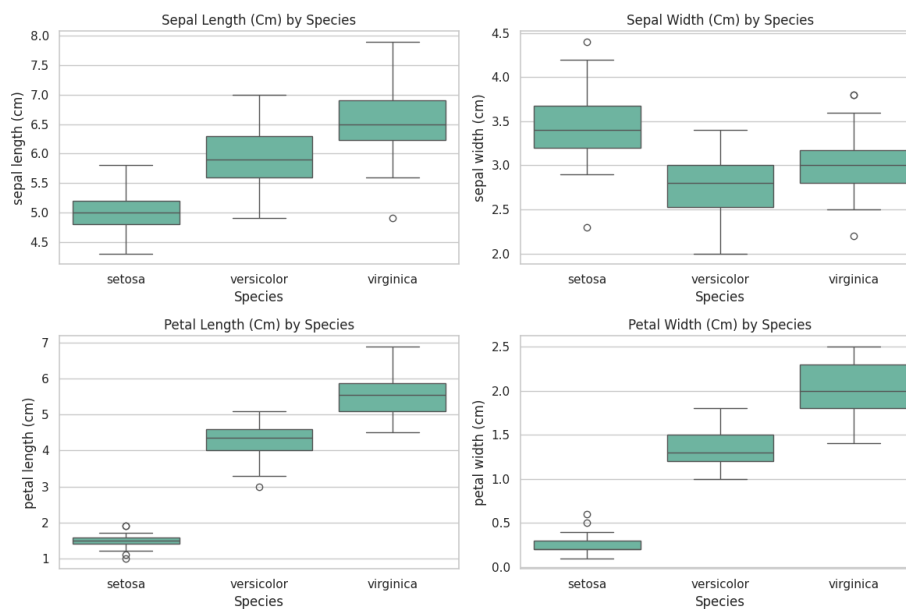
      sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
count              150.000000           150.000000           150.000000           150.000000
mean                5.843333             3.057333             3.758000             1.199333
std                 0.828066             0.435866             1.765298             0.762238
min                 4.300000             2.000000             1.000000             0.100000
25%                 5.100000             2.800000             1.600000             0.300000
```

```
# Class distribution.  
df["species"].value_counts()
```

	count
species	
setosa	50
versicolor	50
virginica	50

dtype: int64

4. Visualisations



6. Train/Test Split

```
# Separate features and target.
x = df[feature_columns]
y = df["species"]

# Use an 80/20 split and stratify to preserve class proportions in train and test sets.
X_train, X_test, y_train, y_test = train_test_split(
    x,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)

X_train.shape, X_test.shape, y_train.shape, y_test.shape

((120, 4), (30, 4), (120,), (30,))
```

	Model	Accuracy
1	K-Nearest Neighbours	1.000000
0	Logistic Regression	0.966667
2	Decision Tree	0.933333
3	Random Forest	0.900000

Model Evaluation:

=====

Logistic Regression

Accuracy: 0.9667

Confusion Matrix:

```
[[10 0 0]
 [ 0 9 1]
 [ 0 0 10]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	0.90	0.95	10
virginica	0.91	1.00	0.95	10
accuracy		0.97		30
macro avg	0.97	0.97	0.97	30
weighted avg	0.97	0.97	0.97	30

=====

K-Nearest Neighbours

Accuracy: 1.0000

Confusion Matrix:

```
[[10 0 0]
 [ 0 10 0]
 [ 0 0 10]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	10
virginica	1.00	1.00	1.00	10
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

=====

Decision Tree

Accuracy: 0.9333

Confusion Matrix:

```
[[10 0 0]
 [ 0 9 1]
 [ 0 1 9]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

=====

Random Forest

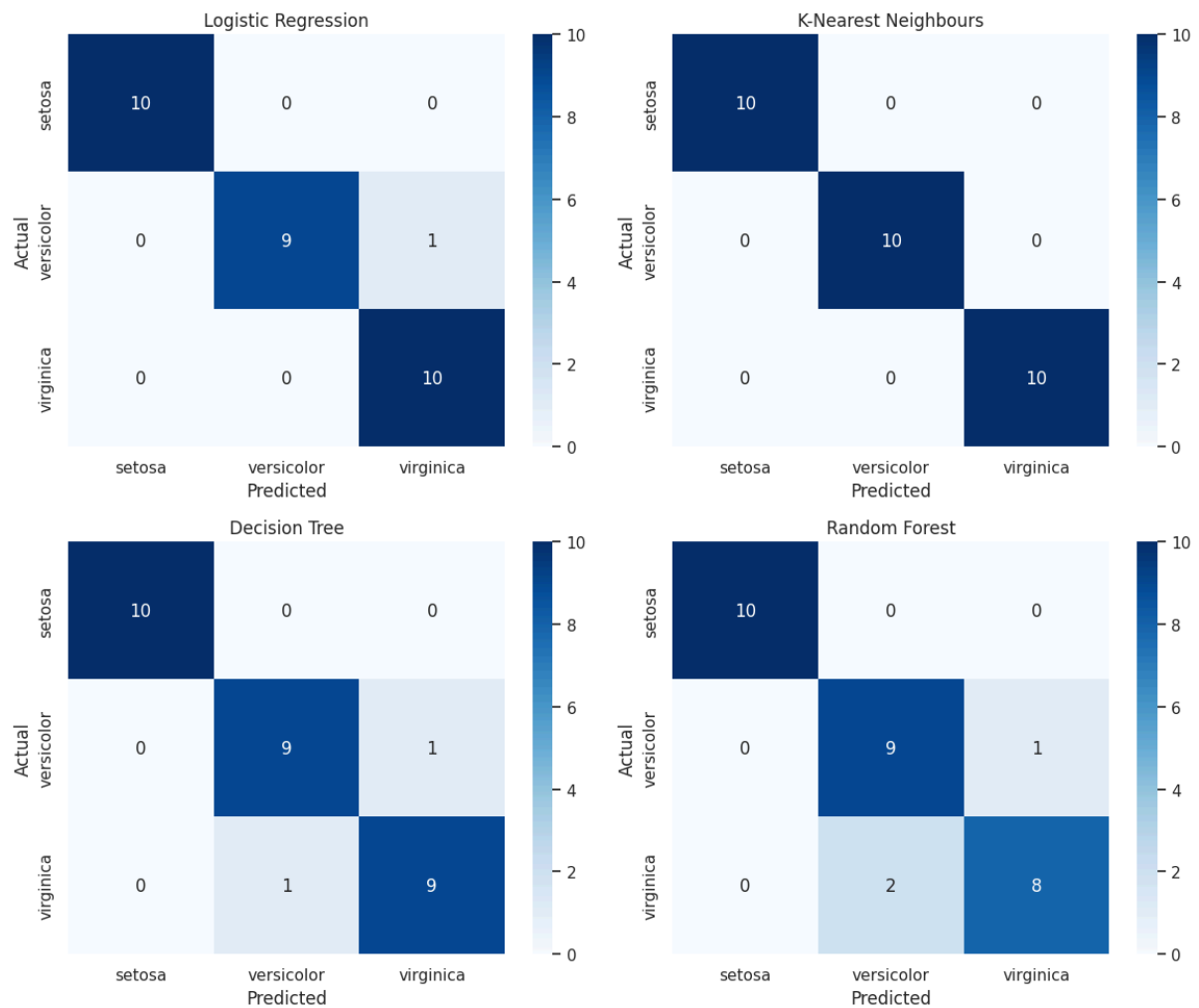
Accuracy: 0.9000

Confusion Matrix:

```
[[10 0 0]
 [ 0 9 1]
 [ 0 2 8]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.82	0.90	0.86	10
virginica	0.89	0.80	0.84	10
accuracy		0.90		30
macro avg	0.90	0.90	0.90	30
weighted avg	0.90	0.90	0.90	30



Best model(s):

	Model	Accuracy
--	-------	----------

1	K-Nearest Neighbours	1.0
---	----------------------	-----

The best-performing model(s) achieved an accuracy of 1.0000. The final choice should prefer the simplest tied model when performance is equal, because simpler models are easier to explain and maintain.